

Relatório de integração com MOM — Sprint 2

O que essa sprint pediu

A Sprint 2 do enunciado pede pra integrar um middleware orientado a mensagens no backend e usar essa camada pra notificar o prestador de forma assíncrona. O item 2.2 é direto: arquitetura orientada a eventos, e o app do prestador não pode ficar fazendo polling pra ver se chegou pedido novo.

Acrescentei um requisito meu na execução: a notificação tem que ser durável. Ou seja, se o processo que escuta a fila estiver fora do ar no momento em que o evento for publicado, a mensagem precisa esperar e ser entregue assim que ele voltar. Isso não está escrito assim no enunciado, mas qualquer sistema de mensagens de verdade tem essa propriedade, e eu queria que o meu também tivesse.

A escolha do broker

A primeira versão foi com Redis Pub/Sub. O setup é barato, o cliente Node (`ioredis`) é fácil, e parecia atender o que o enunciado pedia. Aí descobri o problema: `PUBLISH` no Redis é fire-and-forget. Se ninguém estiver inscrito no canal naquele instante, a mensagem é descartada e nunca volta. Isso quebra o requisito de durabilidade que eu tinha colocado.

Troquei pra RabbitMQ. Está listado no enunciado como alternativa válida ("RabbitMQ ou Redis Pub/Sub") e tem filas duráveis com mensagens persistentes em disco de fábrica. Não precisei inventar nada, só configurar direito.

Subi o RabbitMQ via Docker Compose, num container dedicado ao projeto (`festalink-rabbitmq`). A porta 5672 do host é a porta AMQP onde a API e o worker conectam. A 15672 é a UI de admin, que foi muito útil pra debugar durante o desenvolvimento: dá pra ver as filas, conexões, contadores de mensagens, tudo em tempo real. O usuário/senha que configurei é `festalink/festalink`.

A topologia que escolhi é um exchange `topic` durável chamado `festalink.eventos`, com uma fila durável `festalink.worker` bindada nele pela routing key `reserva.#`. Cada evento entra no exchange com routing key derivada do tipo (`ReservaSolicitada` vira `reserva.solicitada`, `ReservaAprovada` vira `reserva.aprovada`). Publico com `persistent: true` e uso `createConfirmChannel` no producer, então a API só considera a publicação concluída quando o broker confirma que recebeu. No consumer rodo com `prefetch(1)` e `ack` manual, e se der erro no processamento faço `nack` com requeue.

Considereei usar um exchange `direct` em vez de `topic`, mas o `topic` deixa mais fácil adicionar novos tipos de evento depois sem mudar o binding (basta seguir o padrão `reserva.*`). Pra usar o `amqplib` (versão Promise, não a com callback).

A separação em três processos

Pra deixar o desacoplamento entre produtor e consumidor visível no desenho, dividi o backend em dois processos Node que se comunicam só pelo broker. Na versão anterior era tudo um único processo Node que publicava e consumia ao mesmo tempo. Funcionava, mas dá pra fingir que produtor e consumidor são a mesma coisa quando eles moram no mesmo executável, e não são.

Os três processos do sistema ficaram:

- A API (`node src/api.js` , porta 3000) tem o Express com todas as rotas REST, lê e escreve no SQLite, e publica eventos no RabbitMQ. Não consome nada, não tem WebSocket.
- O RabbitMQ é o container do Docker na porta 5672. Recebe mensagens da API e entrega ao worker.
- O Worker (`node src/worker.js` , porta 3001) consome eventos do RabbitMQ e mantém o servidor WebSocket aberto pra empurrar notificações pro app do prestador. Sem rotas HTTP.

Os dois processos Node compartilham o código de domínio, use cases e repositórios. A diferença está só na composição da infra: o `composition.js` agora tem duas funções, `comporApi()` e `comporWorker()` , que instanciam só o que cada lado precisa. Se um dia eu quisesse rodar a API numa máquina e o worker em outra, daria pra fazer sem nenhuma mudança no código, só apontando os dois pra um RabbitMQ acessível.

Na prática isso funciona bem. Eu posso derrubar o worker sem afetar a API, e a recíproca também vale.

Os adapters

Na camada externa entraram dois adapters novos. Cada um implementa uma porta da camada de use cases:

- O `RabbitMQEventPublisher` faz `connect` , abre um `confirmChannel` , declara o exchange e publica os envelopes JSON. A routing key é derivada do `evento.type` por uma função simples que converte PascalCase pra dot-snake.
- O `RabbitMQEventConsumer` faz `connect` , abre um channel normal, declara o exchange e a fila, binda com as routing keys que vêm do `.env` , configura `prefetch(1)` , registra o consumer com ack manual e despacha cada mensagem pra um callback configurado no boot.

O `WebSocketNotifier` já existia da versão anterior e não precisou mudar. Ele mantém um Map de `prestadorId → Set<WebSocket>` , valida o JWT no handshake e entrega o envelope ao socket certo via `notificarPrestadorDoSalao` .

Os use cases que publicam eventos (`CriarReserva` , `AprovarReserva` , `RecusarReserva`) recebem `eventPublisher` por injeção e publicam o evento em microtarefa depois do `commit` da transação. Eles não conhecem `amqplib` nem `ws` , usam só a interface `publicar(evento)` . Isso pagou dividendos na hora da migração: os testes Jest dos três use cases passaram sem alteração, porque os mocks também usavam a mesma interface.

A demonstração

Pra mostrar o fluxo, tem um cliente WebSocket de teste em `backend/scripts/ws-client.js`. Ele recebe um JWT de prestador na linha de comando, conecta no worker e imprime cada mensagem que chega.

O roteiro do teste manual foi mais ou menos esse:

1. Subi o RabbitMQ via `docker compose up -d` e esperei o healthcheck passar.
2. Subi API e worker em terminais separados (`npm run dev:api` num, `npm run dev:worker` em outro).
3. Em outro terminal, conectei o `ws-client.js` com o JWT do prestador. Logo chegou `{"type": "Conectado", "prestadorId": N}`, sinal de que a sessão WS está aberta.
4. Em mais um terminal, mandei `POST /reservas` como cliente via curl. A reserva entrou no banco, a API publicou `ReservaSolicitada` no exchange, o worker consumiu, e o evento apareceu no terminal do `ws-client.js` em menos de 10 ms de diferença entre o `criadoEm` da reserva e o timestamp da chegada.
5. Pra validar a durabilidade, derrubei o worker com Ctrl+C, criei mais uma reserva e aprovei a anterior. Olhei a fila pela UI do RabbitMQ em `localhost:15672` e mostrava `messages: 2`, `consumers: 0`. Subi o worker de volta e ele consumiu o backlog imediatamente, a fila zerou, e os dois eventos apareceram nos logs do consumer.

O log do worker durante o teste fica direto:

```
[consumer] ReservaSolicitada salao=6 reserva=7
[consumer] ReservaSolicitada salao=6 reserva=8
[consumer] ReservaAprovada salao=6 reserva=7
```

Testes automatizados

A bateria de testes Jest cobre o comportamento dos use cases que publicam eventos. Os mocks de `eventPublisher` usam só a interface `publicar`, então a troca de Redis pra RabbitMQ passou sem mexer numa linha de teste. Os testes verificam que `CriarReserva`, `AprovarReserva` e `RecusarReserva` chamam `publicar` uma vez no caminho feliz com o `type` e o `payload` corretos, que não publicam nada quando a operação falha por validação ou autorização, e que a publicação acontece depois do commit (a ordem das chamadas é verificada pelo mock do `unitOfWork`). A regressão da Sprint 1 continua passando inteira.

Limitações que eu reconheço

Duas coisas ainda não estão no estado em que ficariam num sistema de produção.

A primeira é que a publicação do evento não está na mesma transação do INSERT. Se a API morrer entre o `commit` do SQLite e o `publish` ao broker, o evento é perdido: o cliente vê a reserva no banco, mas o prestador nunca recebe a notificação. Mesmo com o channel em `confirm`, o `confirm` só me ajuda depois que o `publish` foi chamado. Pra resolver isso direito o caminho é padrão Outbox: persistir o evento na mesma transação numa tabela `outbox` e ter um worker dedicado lendo essa tabela e empurrando ao broker. Pro escopo da disciplina aceitei a janela de risco e deixei anotado aqui.

A segunda é que a durabilidade vai do broker até o worker, mas não vai do worker até o app. Se o app do prestador estiver offline quando o worker processar a mensagem, o `WebSocketNotifier` tenta entregar pra zero sockets e o worker faz `ack` do mesmo jeito. A mensagem some do ponto de vista do usuário final. Pra cobrir isso seria push notification (FCM) ou um endpoint REST de notificações pendentes que o app consulta quando reconecta. Fica fora do escopo da Sprint 2 mas é um próximo passo natural.

Próximos passos

A Sprint 3 vai trazer o app cliente Flutter, que consome a REST com polling. O item 3.4 do enunciado libera polling pro cliente, então não precisa de WebSocket desse lado. A Sprint 4 vai trazer o app prestador, e é ele que vai abrir o WebSocket descrito aqui. Quando os dois apps estiverem no ar, o fluxo ponta-a-ponta fica visível: cliente solicita pelo app, prestador recebe push em tempo real via WS, aprova pelo app, e o cliente vê a aprovação no próximo polling.